Bitcoin Conference i18n Pipeline

Summer of Bitcoin 2026 Proposal

Ayash Bera

Ayash Bera

ayashbera@gmail.com | +91-9874099369 | GitHub: Ayash-Bera

Kolkata, India | UTC+5:30

Techno Main Salt Lake, B.Tech Information Technology (2023–present)

Synopsis

Bitcoin conferences produce some of the most substantive public discussion happening in finance. The arguments for sovereign adoption, self-custody law, Bitcoin as a long-run savings vehicle — that reasoning gets built and stress-tested at these events. It is also entirely in English, which means it is inaccessible to the majority of the world's Bitcoin holders.

This project builds a production pipeline that downloads Bitcoin conference audio from YouTube, transcribes it using multiple STT providers, translates the transcript into Indian regional languages starting with Hindi, scores accuracy where ground truth exists, and exports structured data for the Genesis KB knowledge base. The competency test proves the pipeline works end to end on real conference audio. The summer project expands it to cover all ten major Bitcoin 2025 conference talks, adds Tamil, Telugu, Bengali, and Marathi as target languages, builds a browseable web frontend for the translated transcripts, and contributes the i18n layer directly to the Genesis KB codebase.

Competency Test

The competency test deliverable is a working STT evaluation pipeline that runs on real Bitcoin 2025 conference audio and produces English transcripts, Hindi translations, and structured CSV output. I built providers for Sarvam (saarika:v2.5), Gemini (gemini-2.0-flash), and OpenAI Whisper-1, a WER/CER scoring module against bitcointranscripts.com ground truth, a Hindi translation step using Sarvam's mayura:v1 model, and an optional correction and summarization pass via LLM.

Source: https://github.com/Ayash-Bera/braidpool

The pipeline ran successfully on Vivek Ramaswamy's 12-minute keynote from Bitcoin 2025, "Ending America's Crisis." Sarvam transcribed 729 seconds of audio into 10,693 characters of English text with no hallucinations. The translation step split that into 14 chunks and returned a complete Hindi translation. Bitcoin-domain terms came through correctly: "Bitcoin" as ■■■■■■■,

"compound interest" as ■■■■■■■■■■ ■■■■■, "self-custody" as ■■■■-■■■■■■■.

Results (live links):

- English transcript — Vivek Ramaswamy, Bitcoin 2025 (10,693 chars, 729s)
- Hindi translation — same talk, via Sarvam mayura:v1 (14 chunks)
- Full evaluation CSV — all columns including WER/CER fields

The output CSV row for that video:

Column	Value
video_slug	bitcoin-2025-vivek-ramaswamy
provider	sarvam-stt-v2.5
raw_transcript	full English transcript
hindi_transcript	full Hindi translation
corrected_transcript	(populated when Anthropic key set)
summary	(populated when Anthropic key set)
wer	(auto-computed when bitcointranscripts.com publishes 2025 ground truth)
cer	(auto-computed when bitcointranscripts.com publishes 2025 ground truth)
audio_duration_seconds	729.4

21 of 21 tests pass.

What I Found in the Codebase

Before writing anything I cloned genesis-kb and read the transcription_engine source. The upstream pipeline has real infrastructure dependencies: PostgreSQL for job state, Deepgram for audio transcription, AWS S3 for storage. None of those are available standalone. Running the pipeline without all three fails at import time — several modules call **`boto3.client()`** and **`sqlalchemy.create_engine()`** at the module level, not inside functions, so even loading the provider crashes the process if the environment is not set up.

The vendor directory also ships its own **`pytest.ini`** that sets **`testpaths = .`** and **`rootdir = vendor/transcription_engine`**. When I first ran **`pytest`** from the repo root it picked up that config, found the vendor conftest, and tried to import **`clint`**, which is not installed. The test suite failed before running a single project test. I renamed the vendor **`pytest.ini`** to **`pytest.ini.bak`** and added **`--ignore=vendor/transcription_engine/test`** to the project's **`pytest.ini`**. After that the 21 project tests run cleanly in isolation.

The genesis-kb provider is now behind a **`GENESIS_KB_ENABLED`** environment flag. If the flag is not set it does not import. The flag lets the evaluation pipeline run standalone in CI or on a laptop without any of the infrastructure, while still exercising the real upstream pipeline in an environment where it is configured.

The hardest constraint I hit was Sarvam's 30-second audio limit. Conference talks run between 12 and 77 minutes. Sending a 17MB MP3 directly returns a 422. The fix was ffmpeg chunking: split the audio into 25-second segments, transcribe each chunk, rejoin with a space separator. The boundary problem — a sentence split mid-word across two chunks — does not come up in practice because ffmpeg cuts at the requested offset regardless of audio content, and speech recognition handles partial-word audio at clip boundaries without hallucinating.

Gemini's free tier has a hard **`limit: 0`** for audio workloads, not a soft quota that resets daily. Four different API keys across three Google Cloud projects all returned the same error: **`GenerateRequestsPerDayPerProjectPerModel-FreeTier, limit: 0`**. The pipeline handles this by catching the 429, running exponential backoff starting at 10 seconds, and failing gracefully after five retries. Gemini rows are skipped; Sarvam continues. Enabling billing resolves it without touching the code.

Technical Design

Provider abstraction

Every STT provider implements a two-method interface: a **`name`** string and **`transcribe(audio_path: Path) -> str`**. New providers add one file and one import. The evaluator loop does not change. Provider selection at startup is purely by environment variable: no key, no provider.

```
eval/providers/
■■■ __init__.py        # Provider ABC
■■■ sarvam_stt.py      # saarika:v2.5, ffmpeg chunking
■■■ gemini_stt.py      # gemini-2.0-flash, exponential backoff
■■■ openai_stt.py      # Whisper-1
■■■ genesis_kb.py      # upstream pipeline (GENESIS_KB_ENABLED flag)
```

Audio chunking

`_split_audio()` in **`sarvam_stt.py`** runs ffmpeg to cut the input file into 25-second segments, collects the output paths, and returns them. **`_transcribe_chunk()`** posts each path to the Sarvam endpoint with **`language_code: en-IN`** and **`model: saarika:v2.5`**. The full **`transcribe()`** method joins the chunk transcripts with a space. If any chunk fails with a non-200 status it raises **`RuntimeError`** with the status code and response body, which the outer evaluator catches and logs per provider.

Translation

`eval/translator.py` splits the English transcript into 900-character chunks on sentence boundaries (split on **`.`**, **`!`**, **`?`**) and posts each to **`https://api.sarvam.ai/translate`** with **`source_language_code: en-IN`**, **`target_language_code: hi-IN`**, **`model: mayura:v1`**, and **`mode: formal`**. The translated chunks are joined and written to the **`hindi_transcript`** CSV column. The same chunking function works for any target language supported by the Sarvam API; adding Tamil means changing one string.

Accuracy scoring

`eval/metrics.py` computes WER and CER using **`jiwer`** against ground truth fetched from the bitcointranscripts.com GitHub repository. The fetcher caches responses under **`data/ground_truth/`**. For 2025 conference content the ground truth slugs are **`None`** — the community has not published transcripts yet. WER and CER are left blank and populate automatically on the next run once the ground truth exists.

Incremental CSV writes

The first version of **`run_eval.py`** collected all rows and wrote them at the end. A 77-minute video failing halfway through meant losing all prior data for that run. The current version opens the CSV file at the start of the run, writes and flushes after each video, and keeps the file handle open until all videos are done. Partial runs always produce readable output.

Correction and summarization

When **`ANTHROPIC_API_KEY`** is set, **`eval/correction.py`** posts the raw transcript to **`claude-sonnet-4-6`** with a Bitcoin-domain correction prompt (max 8192 tokens) and **`eval/summarizer.py`** produces a 200-word domain summary. Both are skipped silently if the key is absent. The corrected transcript goes into **`corrected_transcript`**; the summary goes into **`summary`**.

Timeline

Week 1: Codebase audit and ground truth pipeline

Read every file in genesis-kb's transcription_engine that touches transcription, language handling, and output formatting. Write up what I find before touching any code. Map where an i18n output stage would attach to the existing job pipeline. Run the full evaluation on all 10 Bitcoin 2025 videos and confirm output.

Week 2: Tamil and Telugu providers

Add Tamil (**`ta-IN`**) and Telugu (**`te-IN`**) as target languages in **`translator.py`**. Run both on the existing 10 transcripts. Check that Sarvam's mayura:v1 handles Bitcoin terminology in both scripts — specifically ■■■■■■■■ (Hindi), ■■■■■■■■■■ (Tamil), ■■■■■■■■■■■ (Telugu). Fix any terminology drift in the translation output.

Weeks 3–4: Bengali and Marathi

Add Bengali (`bn-IN`) and Marathi (`mr-IN`). The CSV schema grows by three columns (`tamil_transcript`, `telugu_transcript`, `bengali_transcript`, `marathi_transcript`). Run the full 10-video corpus through all five language outputs and verify no chunk boundary artifacts appear in any language.

Weeks 5–6: Web frontend

Build a Next.js frontend that reads from the CSV and presents transcripts by language, conference, and speaker. No database — reads from flat files served via the Next.js API routes. Search by speaker name or conference. Each talk page shows the English transcript and a language switcher for the available translations. Deploy to Vercel. This is the browseable interface that makes the data useful to non-developers.

Week 7: WER/CER tracking

Bitcointranscripts.com publishes new transcripts irregularly. Write a GitHub Actions workflow that runs weekly, checks for new ground truth entries for the 10 videos, runs the accuracy scoring if ground truth is now available, and commits the updated results to the repo. When ground truth exists the scoring runs automatically; nothing has to be triggered manually.

Week 8: Genesis KB integration

Contribute the i18n output stage to the genesis-kb codebase. The translation step runs after the main transcription job completes, reads the transcript from the job output, posts it to the translation API, and writes the result back to the job record. This is a new output handler, not a modification to the transcription path. Write tests for the new handler using the same mock patterns as the existing test suite.

Weeks 9–10: OpenAI Whisper comparison

Add an OpenAI API key and run Whisper-1 across all 10 videos. Compare WER and CER between Sarvam and Whisper-1 for the talks where bitcointranscripts.com ground truth exists. Document where each provider diverges: proper nouns (speaker names, company names), Bitcoin-specific terms, numbers. The comparison data goes into the proposal and into the Genesis KB evaluation report.

Week 11: More conferences

Extend the video list to cover Baltic Honeybadger 2025 and Bitcoin MENA 2025 talks beyond the ones already included. Target 25 total videos. Run the full pipeline on the expanded corpus and verify no provider-specific failures appear on longer talks (60+ minutes).

Week 12: Buffer and cleanup

Buffer for review feedback, flaky provider runs, and anything that took longer than expected. Write up what the Gemini quota constraint actually costs at scale: how many videos can be processed

per day on the free tier versus a paid tier, and what the realistic operational budget looks like for a project that wants to cover every Bitcoin conference talk published in a year.

I have no major exam commitments during this period and can work full-time hours.

After the Summer

The language list is not exhausted. Sarvam supports Gujarati, Punjabi, Odia, and Kannada in addition to the five this project adds. Each is one parameter change in the translation call. The frontend language switcher already handles arbitrary language codes; adding a new one means running the translation and deploying a new CSV.

The Genesis KB integration, once merged, means future transcription jobs automatically produce multilingual output without any additional pipeline work. Every new talk that enters the knowledge base gets translated alongside the English transcript.

The WER tracking workflow continues after the summer. As bitcointranscripts.com publishes 2025 ground truth the accuracy numbers fill in automatically. If Sarvam releases a new model the comparison runs cleanly against the same benchmark.

Why This Matters

The Bitcoin 2025 conference had talks from Vivek Ramaswamy, Jack Mallers, Ross Ulbricht, Michael Saylor, and JD Vance. Those talks cover the legal case for self-custody, the compound interest argument for Bitcoin as a generational savings vehicle, and the political trajectory of Bitcoin policy in the United States. None of it is available in any Indian language.

India has roughly 600 million Hindi speakers and one of the fastest-growing Bitcoin communities in the world. The Ramaswamy speech alone — his argument that Bitcoin should be the hurdle rate for risk investments, that a $10,000 investment compounding at 15% annually becomes $2.6 million by age 40 — is exactly the kind of financial reasoning that people in India are actively looking for and cannot access because it was delivered in English at a conference in Las Vegas.

This pipeline does not summarize that talk. It produces the full transcript in Hindi, every word, from "It's good to be back at the Bitcoin conference" through the compound interest slides to "Keep fighting and we'll get it done." A Hindi speaker can read what Ramaswamy actually said, not a filtered version of it.

The WER and CER metrics matter too. Running the same audio through Sarvam and Whisper-1 and measuring accuracy against ground truth produces real data on which provider handles Bitcoin-specific English better. That data is useful to anyone building transcription into a Bitcoin application and currently does not exist in a published form.

About Me

I am a third-year IT student at Techno Main Salt Lake in Kolkata. I have been working in the crypto industry for over 1.5 years at L1beat.io. I also founded Avacado, an on-chain balance encryption protocol where users prove sufficient funds via ZK proofs without revealing amounts.

Building the competency test taught me things I did not expect. The vendor pytest.ini problem only showed up when I ran the test suite from the repo root — nothing documented it, and the error message pointed at a missing `clint` package rather than at the rootdir conflict. Figuring out that Sarvam's 30-second limit applies to the full request payload, not just the audio duration field, cost me a failed API call before I read the error response carefully enough to understand what was being rejected. The Gemini `limit: 0` response looks like a transient rate limit until you see it on four different API keys in three different projects, at which point you realize it is a project-level billing configuration issue and not something that retrying will fix.

I understand the Sarvam STT and translation APIs at the level where I debugged chunk boundary behavior in production audio, not the level where I read the documentation. The pipeline works on real conference audio because I ran it on real conference audio and fixed what broke.